

Informed Search

(Heuristic search techniques)

Uninformed vs Informed Search

- In uninformed search, one can only know whether it is a goal state or not. But, cannot say whether a non-goal state is better than another non-goal state. Therefore, it's a kind of “**blind search**” as we continue to search with a hope that the goal state can be reached, if it exists.
- But, in case of informed search one can evaluate how good/desirable a state is compared to other states with the help of additional information specific to the problem.

Heuristic Search:

Greek word: “Heuriskein” -To discover

Archimedes: “Eureka”- I have found

Observation: “Human being are not optimizers rather satisfiers”.

We don't always look for the “Best solution” but yet satisfied with a “good approximated solution”.

Ex:- i) Looking for a seat in a crowded bus.

ii) Looking for a parking slot.

Choice of Heuristic (Rule of thumb) depends on the problem characteristics.

Design of Heuristics/Heuristic Function(HF) :

- A HF is a function which maps from problem state descriptions to measures of desirability.
- A HF is designed by assigning appropriate weightage to the significant aspects of a problem state.

Issues:

- How to formulate an effective HF?
- What weightage to be given to which parameter?
- Deciding on whether to maximize or minimize the HF value, which depends on the nature of the problem.

Aim:

The purpose of a HF is to guide the search process in the most profitable direction in the presence of multiple alternatives.

Observation:

There is a trade-off between the cost of evaluating a HF & the savings in search time that the HF provides.

Heuristic Search Techniques

Generate and Test(GAT):

Algorithm:

1. Generate a possible solution.
2. Test if it is the desired solution.
3. If solution found then Quit else goto step1.

Remark:

- If solutions are generated systematically, a solution can be obtained(if exists),but takes longtime.
- If solutions are generated randomly, no guarantee for a solution but sometimes a solution can be found in less time.

Ex: “**British Museum Algorithm**”

(Searching of an item in an Unordered List)

Possible Approach:

Search systematically but ignore a path if found unlikely to lead to a solution.

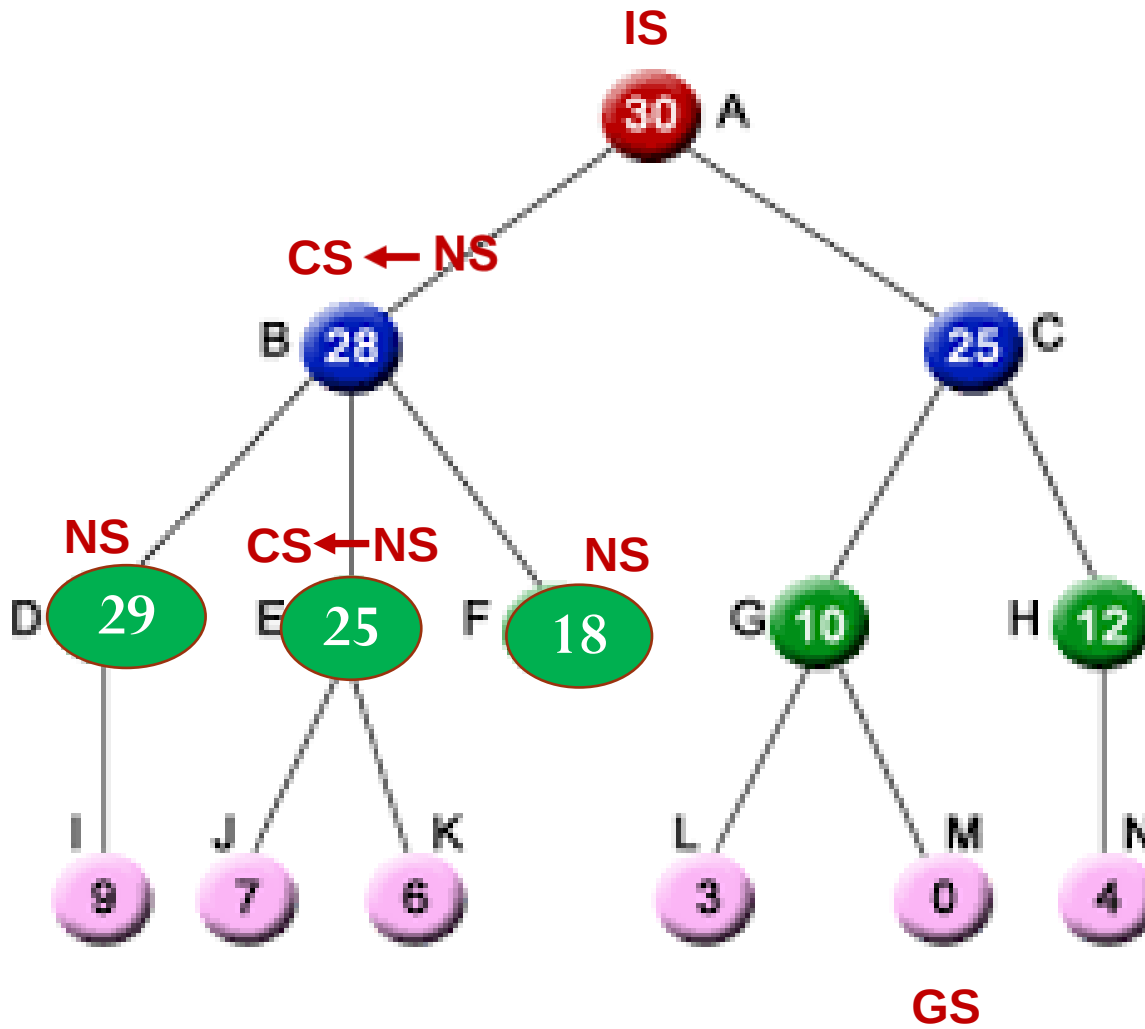
A path can be evaluated by a suitable “Heuristic Function”.

Generate-And-Test” may be modified to “PLAN-GENERATE-TEST”, while planning use some “Constraint-satisfaction techniques”.

HILL Climbing

Algorithm: Simple Hill Climbing

1. Evaluate the Initial State(**IS**):
 - if it is a Goal State(**GS**) then Return Solution & Quit.
 - else consider the **IS** as the current state & proceed further.
 2. Loop until a solution is found or there are no new operators to be applied in the current State(**CS**).
 - A) Select an operator(**op**) that has not yet been applied to the **CS**.
Apply it to generate a New State(**NS**).
 - B) Evaluate the **NS**
 - i) If **NS** is a **GS**, then return Solution & Quit.
 - ii) If not but **NS** better than the **CS** then consider that as the **CS** (**CS**←**NS**) & proceed
 - iii) If **NS** is not better than **CS** then continue in the loop by selecting the next applicable **operator** for **CS**.
- [End of loop]
- (Trying to locate a place in a big unfamiliar city w.r.t a tall building.)



Steepest-Ascent Hill Climbing

This approach evaluates all alternatives at a **CS** & selects the best one as the **NS**. It is also called “**Gradient Search**”.

Algorithm:

1. Evaluate the **IS**:

If it is a **GS** then Return Solution & Quit.

else proceed with the **IS** as the **CS**.

2. Loop until a solution is found or a complete iteration produces no change to the **CS**.

A) Apply a valid operator to **CS** & generate a **NS** and set it to **succ** ($\text{succ} \leftarrow \text{NS}$)

B) For each applicable op to **CS** do:

i) Apply op to **CS** & generate a **NS**.

ii) Evaluate the **NS**

If it is a **GS**, then Return Solution & Quit.

Else compare it with **succ**.

If **NS** is better than **succ** then set **succ** to **NS**

[i.e. **succ** ← **NS**]

Else leave **succ** unchanged.

C) If **succ** is better than the **CS** then

set **CS** to **succ** [i.e. **CS** ← **succ**]

i.e. move to the next best state.

[End of loop]

Drawbacks of Hill Climbing Algorithm:

HC may fail to find a solution when the algorithm terminates by reaching a state from where no better states can be generated.

This can happen when the HC algorithm reaches either a “**Local maximum**” or “**Plateau**” or a “**Ridge**”.

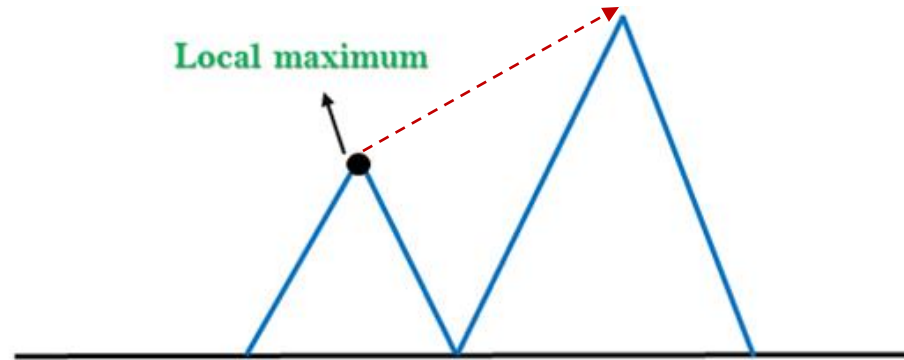
Observation:

The HC algorithm suffers from 3 drawbacks as it uses only local info, rather it must use global info.

Local Maximum:

“Foot Hills”

It is a state in the search space which is better than all its neighbors but is not better than some state further away.

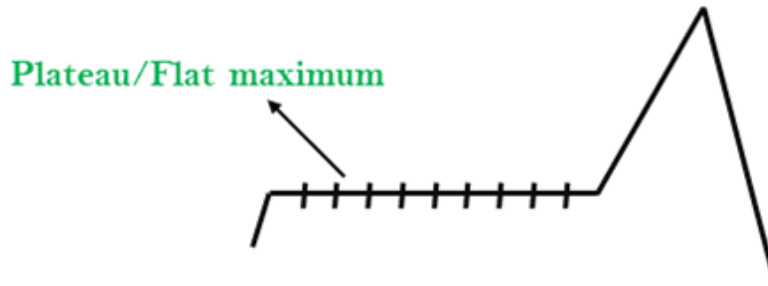


Solution:

Maintain a list of promising paths so that the algorithm can backtrack to an ancestor in the search space and explore other alternate paths.

“Plateau”

It is a flat area in the search space in which all the neighboring states have the same HF value.



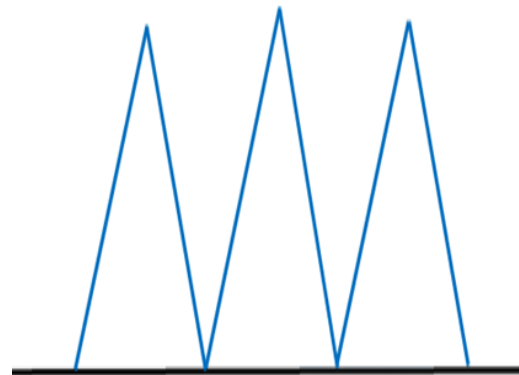
i.e. It is not possible to find the best direction by making local comparisons.

Solution:

- Take big steps, that is expand few generation ahead to move to a different section of the search space.
- Randomly select a state which is far away from the current state in order to move to a non-plateau region.

Ridge: (a type of “Foot Hill”)

A ridge is a special form of the local maximum. It is an area in the search space which is higher than its surrounding but itself has slopes, and cannot be reached in a single move as no such operator is available.



Solution:

Apply several operators before doing the evaluation. Move in different directions until you come out of the ridge.

Different regions in the state space



Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state in the state space

Flat local maximum/Plateau: It is a flat space in the state space where all the neighbor states of a current state have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Ridges: Sharp slopes

Local maxima, Plateau, Ridge Problems

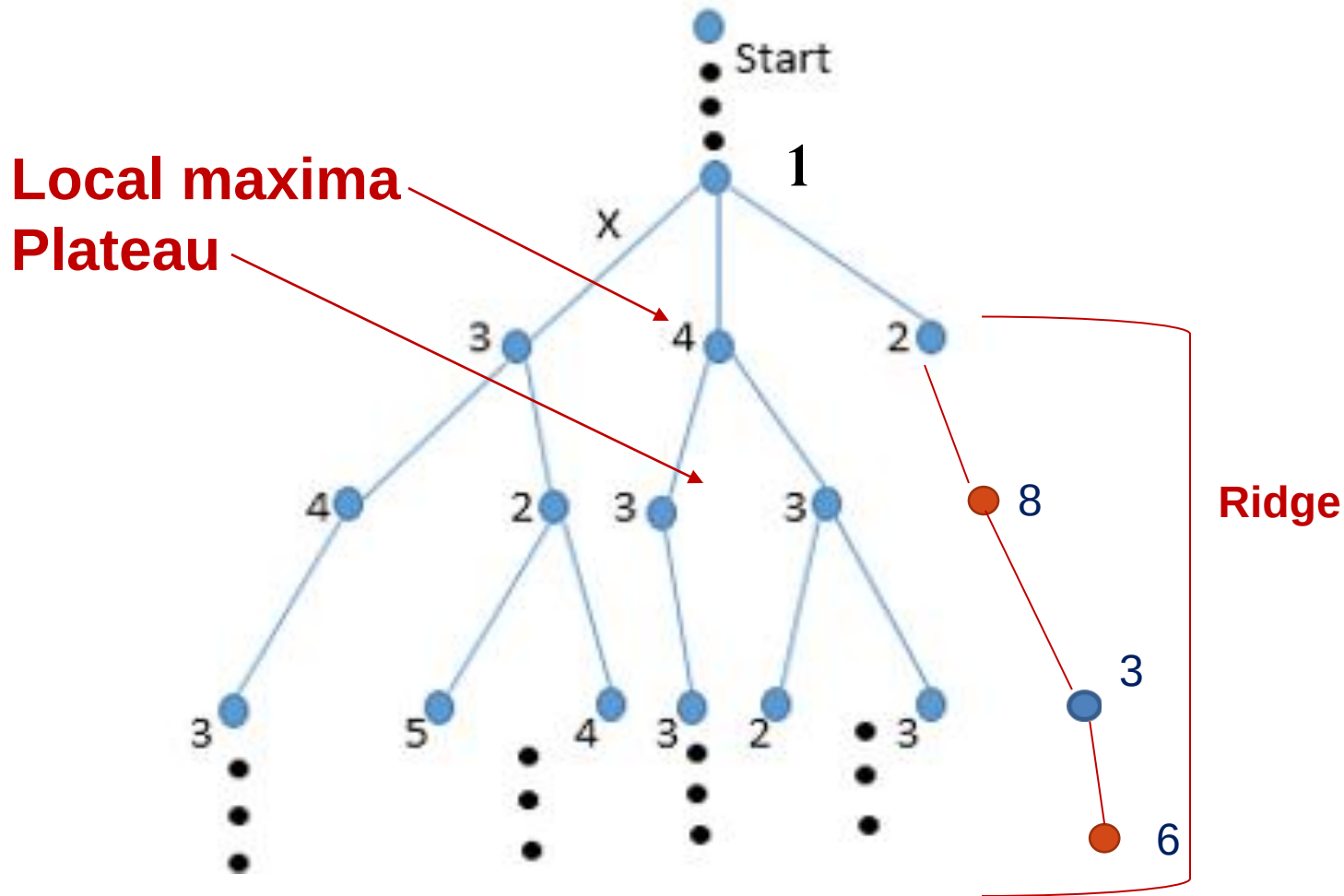
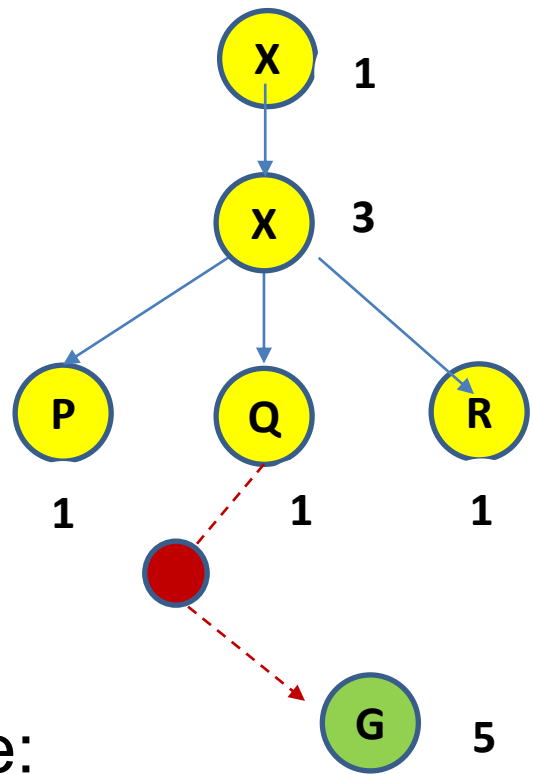
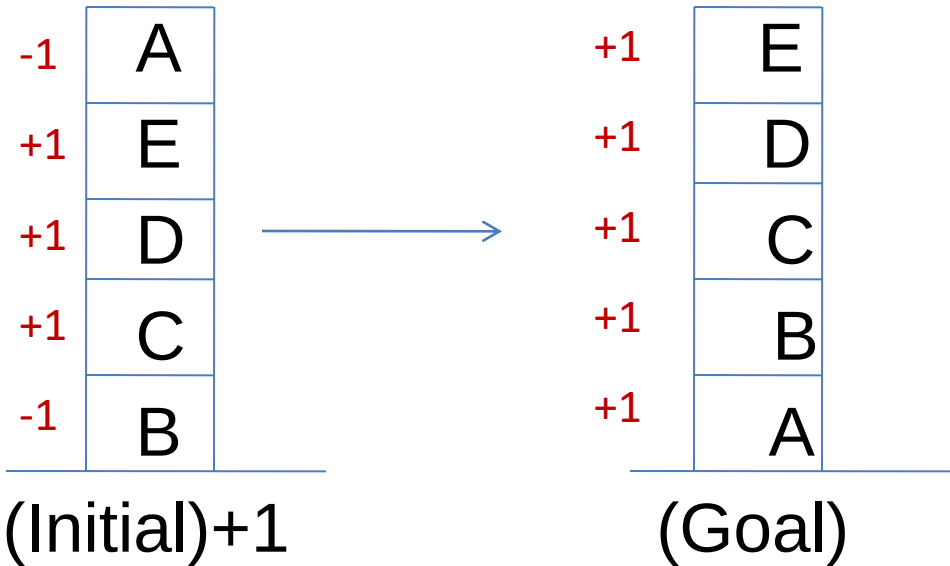
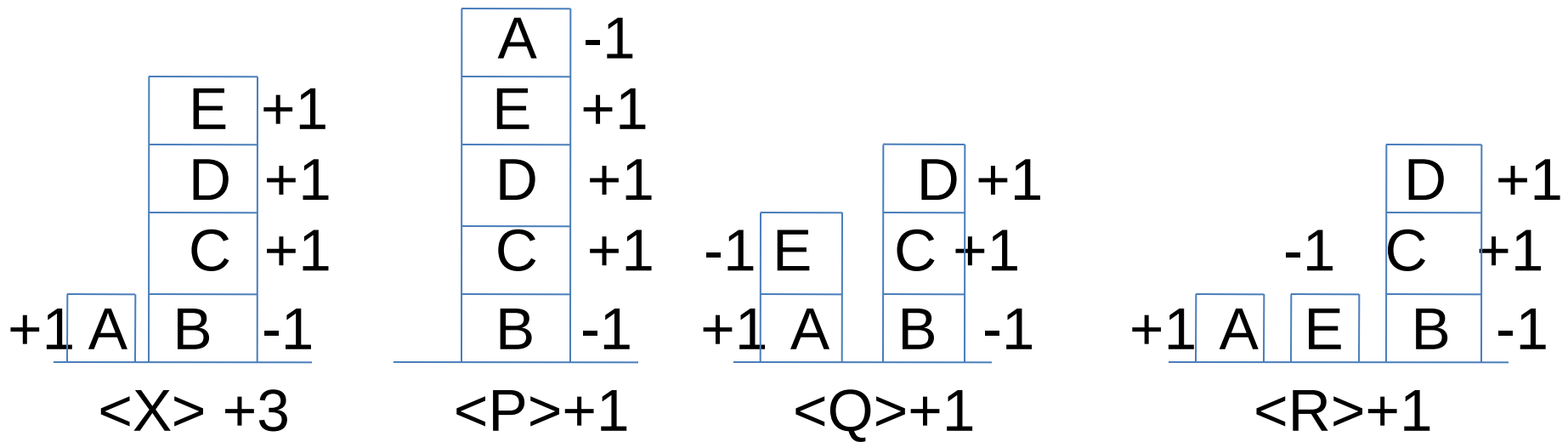


Illustration:



3 possible moves from the NS 'X' are:



Heuristic function used:

1. Assign +1 for every block correctly placed w.r.t. the Goal state.
2. Assign -1 for every block misplaced under a specified block.

Conclusion:

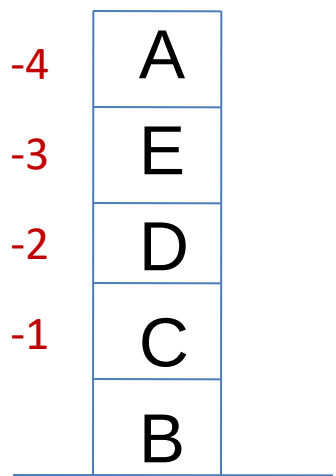
If one uses only local information there is possibility of getting stuck in the state space while moving from an Initial state to a Goal state.

Modified Heuristic

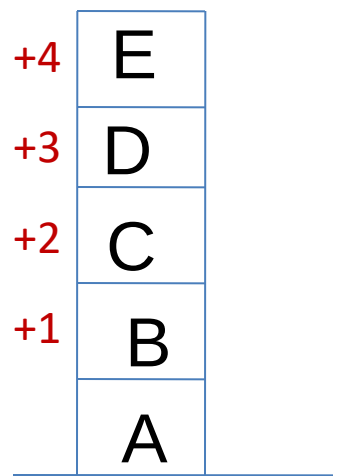
Strategy:

- Incorrect structures are useless, hence ignore them.
 - Correct structures are useful, hence build up on them.
1. Assign +1 for each of the blocks correctly placed in the support structure w.r.t. the Goal state.
 2. Assign -1 for each of the blocks misplaced in the support structure under a specified block.

Illustration:

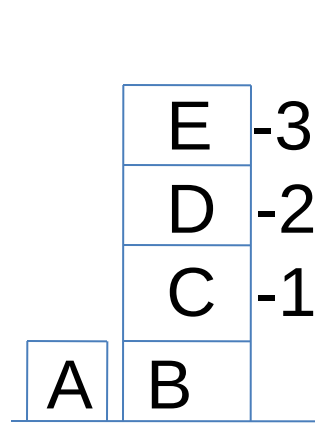


(Initial)-10

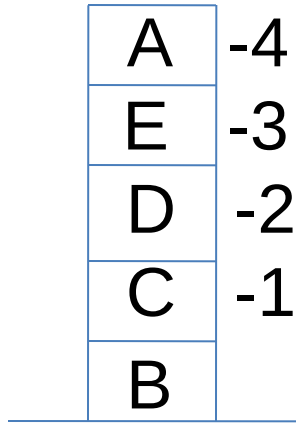


(Goal)

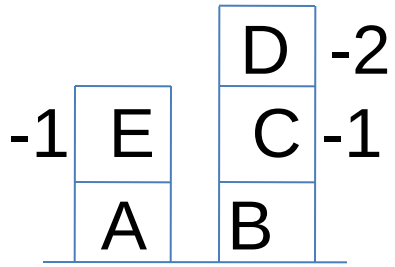
3 possible moves from the NS 'X' are:



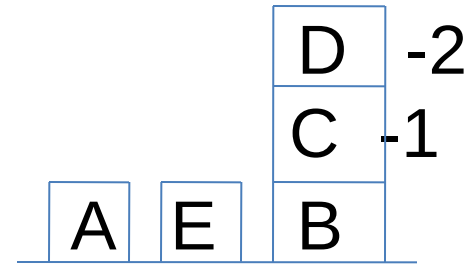
<X> - 6



<p>-10



<Q>-4



<R>-3

